

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	P.Harsha vardhan	Reg.No.:	192525464
Department:	BTECH-AIML	Date:	11-8-2026

Code of Conduct:

I P.Harsha Vardhan(192525464) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	----	---	--	--	----------------------------------

Q1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Step 1: Identify the functional dependencies

- $SID \rightarrow SName$ (each student ID determines exactly one student name)
- $CID \rightarrow CName, Instructor$ (each course ID determines one course name and one instructor)
- $SID, CID \rightarrow Grade$ (a grade exists only for a specific student–course pair)
-

Step 2: Determine the candidate key

SID alone cannot determine Grade or CID; CID alone cannot determine SID. Only the composite attribute set {SID, CID} functionally determines every other attribute in the relation.

Candidate Key = { SID, CID }

SQL Demonstration

The dataset below has two entries for SID 101 (different CIDs) and two rows sharing CID C101. The DISTINCT projections confirm $SID \rightarrow SName$ and $CID \rightarrow (CName, Instructor)$ hold consistently, and the GROUP BY confirms no (SID, CID) pair repeats — proving {SID, CID} is a valid key. In practice this is why a marks table always needs both the student and the course as key columns — a single ID by itself is never enough to look up one grade.

```

sqlite> -- Q1 - STUDENT_COURSE: functional dependency and key check
sqlite> CREATE TABLE STUDENT_COURSE (SID INTEGER, SName TEXT, CID TEXT, CName TEXT, Instructor TEXT, Grade TEXT);

sqlite> INSERT INTO STUDENT_COURSE VALUES (101,'Arun','C101','DBMS','Dr. Rao','A'),
...> (101,'Arun','C102','OS','Dr. Iyer','B'),
...> (102,'Bala','C101','DBMS','Dr. Rao','A'),
...> (103,'Chitra','C103','CN','Dr. Rao','C');

sqlite> SELECT * FROM STUDENT_COURSE;
SID | SName | CID | CName | Instructor | Grade
-----+-----+-----+-----+-----+-----
101 | Arun | C101 | DBMS | Dr. Rao | A
101 | Arun | C102 | OS | Dr. Iyer | B
102 | Bala | C101 | DBMS | Dr. Rao | A
103 | Chitra | C103 | CN | Dr. Rao | C

sqlite> SELECT DISTINCT SID, SName FROM STUDENT_COURSE ORDER BY SID;
SID | SName
-----+-----
101 | Arun
102 | Bala
103 | Chitra

sqlite> SELECT DISTINCT CID, CName, Instructor FROM STUDENT_COURSE ORDER BY CID;
CID | CName | Instructor
-----+-----+-----
C101 | DBMS | Dr. Rao
C102 | OS | Dr. Iyer
C103 | CN | Dr. Rao

sqlite> SELECT SID, CID, COUNT(*) c FROM STUDENT_COURSE GROUP BY SID, CID HAVING c > 1;
SID | CID | c
-----+-----+-----
(0 rows)

```

2Q: Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

Step 1: Identify the violation

A relation is in 1NF only if every attribute holds a single atomic value. In the STUDENT relation below, the Courses column stores a comma-separated list — a multi-valued, non-atomic attribute — which violates 1NF.

```

sqlite> -- Q2 (a) - unnormalized STUDENT relation (repeating group)
sqlite> CREATE TABLE STUDENT_UNF (SID INTEGER, SName TEXT, Courses TEXT);

sqlite> INSERT INTO STUDENT_UNF VALUES (1,'Arun','C101,C102,C103'), (2,'Bala','C101');

sqlite> SELECT * FROM STUDENT_UNF;
SID | SName | Courses
-----+-----+-----
1 | Arun | C101,C102,C103
2 | Bala | C101

```

Step 2: Apply 1NF

- Remove the repeating group by giving each (student, course) pair its own row.
- The new primary key becomes the composite {SID, CID}, since SID alone no longer uniquely identifies a row.

```

sqlite> -- Q2 (b) - after 1NF conversion (atomic values, key = SID+CID)
sqlite> CREATE TABLE STUDENT_1NF (SID INTEGER, SName TEXT, CID TEXT);

sqlite> INSERT INTO STUDENT_1NF VALUES (1,'Arun','C101'),(1,'Arun','C102'),
...> (1,'Arun','C103'),(2,'Bala','C101');

sqlite> SELECT * FROM STUDENT_1NF ORDER BY SID;
SID | SName | CID
----+-----+-----
1   | Arun  | C101
1   | Arun  | C102
1   | Arun  | C103
2   | Bala  | C101

```

The table is now in 1NF: every cell holds a single atomic value and rows are uniquely identified by (SID, CID). Splitting the comma-separated list also means a course can now be added or dropped for a student with a single INSERT/DELETE instead of editing a text string.

Q3. $R(A,B,C,D,E)$ with FDs $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$ — Decompose to 2NF

Step 1: Determine the candidate key

Closure of $\{A\}$: $A \rightarrow BC \rightarrow (A,B,C)$; $BC \rightarrow D$ gives (A,B,C,D) ; $D \rightarrow E$ gives (A,B,C,D,E) = all attributes. So A is the sole candidate key, and R is in 1NF (assume atomic attributes).

Step 2: Check for partial dependency

Since the key is the single attribute A (not composite), there is technically no "partial" dependency on part of the key in the classic sense — but the chain $A \rightarrow BC \rightarrow D \rightarrow E$ hides a stronger issue: D and E depend on BC , not directly on the whole key in one step, and BC itself is a non-prime attribute set determined by A . To remove the redundancy this chain introduces, decompose so each determinant becomes its own key.

```

sqlite> -- Q3 (a) - R(A,B,C,D,E), key A, checking BC->D
sqlite> CREATE TABLE R_UNNORM (A INTEGER, B TEXT, C TEXT, D TEXT, E TEXT);

sqlite> INSERT INTO R_UNNORM VALUES (1,'B1','C1','D1','E1'),
...> (2,'B2','C2','D1','E1'),(3,'B1','C1','D1','E1');

sqlite> SELECT * FROM R_UNNORM;
A | B | C | D | E
---+---+---+---+---
1 | B1 | C1 | D1 | E1
2 | B2 | C2 | D1 | E1
3 | B1 | C1 | D1 | E1

sqlite> -- does B,C alone determine D? (A is the whole key)

sqlite> SELECT DISTINCT B, C, D FROM R_UNNORM;
B | C | D
---+---+---
B1 | C1 | D1
B2 | C2 | D1

```

Step 3: Decomposition

- $R_1(A, B, C)$ — key A, holds $A \rightarrow BC$
- $R_2(B, C, D, E)$ — key (B,C), holds $BC \rightarrow D$ and $D \rightarrow E$

This groups attributes so each non-key attribute depends on the whole of its local key, removing the partial-dependency style redundancy. The join below reconstructs the original relation losslessly, so no information from the original R is lost by splitting it this way.

```

sqlite> -- Q3 (b) - 2NF decomposition: R1(A,B,C) and R2(B,C,D,E)
sqlite> CREATE TABLE R1_A_BC (A INTEGER PRIMARY KEY, B TEXT, C TEXT);

sqlite> INSERT INTO R1_A_BC VALUES (1,'B1','C1'),(2,'B2','C2'),(3,'B1','C1');

sqlite> CREATE TABLE R2_BC_DE (B TEXT, C TEXT, D TEXT, E TEXT, PRIMARY KEY(B,C));

sqlite> INSERT INTO R2_BC_DE VALUES ('B1','C1','D1','E1'),('B2','C2','D1','E1');

sqlite> SELECT * FROM R1_A_BC;
A | B | C
---+---+---
1 | B1 | C1
2 | B2 | C2
3 | B1 | C1

sqlite> SELECT * FROM R2_BC_DE;
B | C | D | E
---+---+---+---
B1 | C1 | D1 | E1
B2 | C2 | D1 | E1

sqlite> SELECT r1.A, r1.B, r1.C, r2.D, r2.E FROM R1_A_BC r1 JOIN R2_BC_DE r2 ON r1.B=r2.B AND r1.C=r2.C;
A | B | C | D | E
---+---+---+---+---
1 | B1 | C1 | D1 | E1
2 | B2 | C2 | D1 | E1
3 | B1 | C1 | D1 | E1

```

Q4. 3NF Decomposition — EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

Step 1: Identify the transitive dependency

Candidate key: EmpID. FDs: EmpID → EmpName, DeptID and DeptID → DeptName, ManagerID.

Since DeptID is a non-key attribute that itself determines DeptName and ManagerID, we have a transitive dependency: EmpID → DeptID → DeptName, ManagerID. This means DeptName/ManagerID depend on the key only indirectly, through DeptID — a 3NF violation.

```
sqlite> -- Q4 (a) - EMPLOYEE with transitive dependency EmpID->DeptID->DeptName
sqlite> CREATE TABLE EMPLOYEE_UNNORM (EmpID INTEGER, EmpName TEXT, DeptID TEXT, DeptName TEXT, ManagerID TEXT);

sqlite> INSERT INTO EMPLOYEE_UNNORM VALUES (1,'Ravi','D01','Sales','M01'),
...> (2,'Sneha','D01','Sales','M01'),(3,'Kiran','D02','HR','M02');

sqlite> SELECT * FROM EMPLOYEE_UNNORM;
EmpID | EmpName | DeptID | DeptName | ManagerID
-----+-----+-----+-----+-----
1      | Ravi    | D01    | Sales    | M01
2      | Sneha   | D01    | Sales    | M01
3      | Kiran   | D02    | HR       | M02

sqlite> -- transitive dependency: DeptID determines DeptName, ManagerID

sqlite> SELECT DISTINCT DeptID, DeptName, ManagerID FROM EMPLOYEE_UNNORM;
DeptID | DeptName | ManagerID
-----+-----+-----
D01    | Sales    | M01
D02    | HR       | M02
```

Step 2: Decompose to remove the transitive dependency

- EMPLOYEE(EmpID, EmpName, DeptID) — key EmpID
- DEPARTMENT(DeptID, DeptName, ManagerID) — key DeptID

Now every non-key attribute in each relation depends directly on that relation's key. Rejoining on DeptID reproduces the original data with no loss, and DeptName only needs to be stored once per department instead of once per employee.


```

sqlite> -- Q4 (b) - decomposed to 3NF, rejoined to verify no data lost
sqlite> CREATE TABLE EMP_3NF (EmpID INTEGER PRIMARY KEY, EmpName TEXT, DeptID TEXT);
sqlite> INSERT INTO EMP_3NF VALUES (1,'Ravi','D01'),(2,'Sneha','D01'),(3,'Kiran','D02');

sqlite> CREATE TABLE DEPT_3NF (DeptID TEXT PRIMARY KEY, DeptName TEXT, ManagerID TEXT);
sqlite> INSERT INTO DEPT_3NF VALUES ('D01','Sales','M01'),('D02','HR','M02');

sqlite> SELECT * FROM EMP_3NF;
EmpID | EmpName | DeptID
-----+-----+-----
1     | Ravi    | D01
2     | Sneha   | D01
3     | Kiran   | D02

sqlite> SELECT * FROM DEPT_3NF;
DeptID | DeptName | ManagerID
-----+-----+-----
D01    | Sales    | M01
D02    | HR       | M02

sqlite> SELECT e.EmpID, e.EmpName, d.DeptID, d.DeptName, d.ManagerID FROM EMP_3NF e JOIN DEPT_3NF d ON e.DeptID=d.DeptID;
EmpID | EmpName | DeptID | DeptName | ManagerID
-----+-----+-----+-----+-----
1     | Ravi    | D01    | Sales    | M01
2     | Sneha   | D01    | Sales    | M01
3     | Kiran   | D02    | HR       | M02

```

Q5. BCNF Verification — $R(A,B,C,D)$ with $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$

Step 1: Find candidate keys

Closure of $\{A,B\}$: $AB \rightarrow C$ (given), $C \rightarrow D$ (given), so $AB^+ = \{A,B,C,D\}$ = all attributes. AB is a candidate key.

Closure of $\{C\}$: $C \rightarrow D$, $D \rightarrow A$, so $C^+ = \{A,C,D\}$. Missing B , so C alone is not a superkey — but note C is part of no candidate key by itself; check D similarly: $D^+ = \{A,C,D\}$ ($D \rightarrow A$, and via C ? A doesn't determine C directly) — also not a superkey.

Step 2: Apply the BCNF definition:

BCNF requires every determinant (left-hand side of an FD) to be a superkey. Here, $C \rightarrow D$ holds, but C is NOT a superkey ($C^+ \neq$ all attributes). This is a BCNF violation.

```

sqlite> -- Q5 (a) - R(A,B,C,D), checking C->D against BCNF
sqlite> CREATE TABLE R_ABCD (A TEXT, B TEXT, C TEXT, D TEXT);

sqlite> INSERT INTO R_ABCD VALUES ('A1','B1','C1','D1'),
...> ('A2','B1','C2','D2'),('A1','B2','C1','D1');

sqlite> SELECT * FROM R_ABCD;
A | B | C | D
---+---+---+---
A1 | B1 | C1 | D1
A2 | B1 | C2 | D2
A1 | B2 | C1 | D1

sqlite> -- check candidate key: does AB determine all attributes?

sqlite> SELECT DISTINCT A, B, C, D FROM R_ABCD;
A | B | C | D
---+---+---+---
A1 | B1 | C1 | D1
A2 | B1 | C2 | D2
A1 | B2 | C1 | D1

sqlite> -- check C->D: is C a superkey? (C repeats without determining B)

sqlite> SELECT DISTINCT C, D FROM R_ABCD;
C | D
---+---
C1 | D1
C2 | D2

```

Step 3: Decompose into BCNF

- R1(C, D) — key C, holds $C \rightarrow D$
- R2(A, B, C) — key (A,B), holds $AB \rightarrow C$

In both resulting relations, every determinant is now a superkey of that relation, satisfying BCNF. The join below confirms the decomposition is lossless — note that BCNF is stricter than 3NF precisely because it does not make an exception for determinants that happen to be part of a candidate key.

```

sqlite> -- Q5 (b) - decomposed to BCNF: R1(C,D) and R2(A,B,C)
sqlite> CREATE TABLE R1_CD (C TEXT PRIMARY KEY, D TEXT);
sqlite> INSERT INTO R1_CD VALUES ('C1','D1'),('C2','D2');

sqlite> CREATE TABLE R2_ABC (A TEXT, B TEXT, C TEXT, PRIMARY KEY(A,B));
sqlite> INSERT INTO R2_ABC VALUES ('A1','B1','C1'),('A2','B1','C2'),('A1','B2','C1');

sqlite> SELECT * FROM R1_CD;
C | D
---+---
C1 | D1
C2 | D2

sqlite> SELECT * FROM R2_ABC;
A | B | C
---+---
A1 | B1 | C1
A2 | B1 | C2
A1 | B2 | C1

sqlite> SELECT r2.A, r2.B, r2.C, r1.D FROM R2_ABC r2 JOIN R1_CD r1 ON r2.C=r1.C;
A | B | C | D
---+---+---+---
A1 | B1 | C1 | D1
A2 | B1 | C2 | D2
A1 | B2 | C1 | D1

```

Q6. Lossless-Join Verification Using the Chase (Tabular) Algorithm

Setup

Relation $R(A,B,C)$ decomposed into $R1(A,B)$ and $R2(B,C)$, with the given FD $B \rightarrow C$.

Chase procedure

- Construct a tableau with two rows, one per decomposed relation, using distinct symbols (a_1, b_1, c_1) for $R1$'s row and (a_2, b_2, c_2) for $R2$'s row, matching on shared attributes with the original.
- Apply FD $B \rightarrow C$: wherever two rows agree on B , their C values must be equated. Since B is common to both $R1$ and $R2$, and $B \rightarrow C$ holds, the tableau reduces to an all-'a' row — the lossless-join condition is satisfied.
- Rule: a binary decomposition $R1 \cap R2 \rightarrow R1$ (or $R1 \cap R2 \rightarrow R2$) is lossless iff the common attribute set is a candidate key of at least one of the two pieces. Here B is the key of $R2$, so the join is lossless. If B were not a key of either piece, the same chase would leave two distinct rows agreeing on B , which is exactly what produces spurious tuples on rejoin.

Empirical verification with sample data

Natural-joining $R1$ and $R2$ back together on B reproduces R_ORIG exactly, with no missing and no spurious rows — confirming losslessness.

```

sqlite> -- Q6 - lossless-join check: R1(A,B) join R2(B,C) = R, B is key of R2
sqlite> CREATE TABLE R_ORIG (A TEXT, B TEXT, C TEXT);

sqlite> INSERT INTO R_ORIG VALUES ('A1','B1','C1'),('A2','B1','C1'),('A3','B2','C2');

sqlite> SELECT * FROM R_ORIG;
A | B | C
-----
A1 | B1 | C1
A2 | B1 | C1
A3 | B2 | C2

sqlite> CREATE TABLE R1_AB (A TEXT, B TEXT);
sqlite> INSERT INTO R1_AB SELECT DISTINCT A, B FROM R_ORIG;

sqlite> CREATE TABLE R2_BC (B TEXT PRIMARY KEY, C TEXT);
sqlite> INSERT INTO R2_BC SELECT DISTINCT B, C FROM R_ORIG;

sqlite> SELECT * FROM R1_AB;
A | B
----
A1 | B1
A2 | B1
A3 | B2

sqlite> SELECT * FROM R2_BC;
B | C
----
B1 | C1
B2 | C2

sqlite> -- natural join R1, R2 on B, compare with R_ORIG

sqlite> SELECT R1_AB.A, R1_AB.B, R2_BC.C FROM R1_AB JOIN R2_BC ON R1_AB.B = R2_BC.B;
A | B | C
-----
A1 | B1 | C1
A2 | B1 | C1
A3 | B2 | C2

```

Q7. Multi-Valued Dependencies and 4NF — TEACHER(TID, Subject, Hobby)

Step 1: Identify the MVDs

Subject and Hobby are independent facts about a teacher — knowing a teacher's subjects tells you nothing about their hobbies, yet the design forces every (subject, hobby) combination to be stored for a teacher who has more than one of each. This gives:

- TID $\twoheadrightarrow$ Subject
- TID $\twoheadrightarrow$ Hobby

This is a classic 4NF violation: the relation is already in BCNF (the only FD-based key is the whole relation), but the independent multi-valued facts cause a cross-product style redundancy.

```
sqlite> -- Q7 (a) - TEACHER(TID,Subject,Hobby): MVD cross-product redundancy
sqlite> CREATE TABLE TEACHER (TID INTEGER, Subject TEXT, Hobby TEXT);

sqlite> INSERT INTO TEACHER VALUES (1,'Maths','Chess'),(1,'Maths','Music'),
...> (1,'Physics','Chess'),(1,'Physics','Music');

sqlite> SELECT * FROM TEACHER ORDER BY TID, Subject, Hobby;
TID | Subject | Hobby
-----+-----
1   | Maths   | Chess
1   | Maths   | Music
1   | Physics | Chess
1   | Physics | Music

sqlite> -- every Subject paired with every Hobby for a given TID (cross product)

sqlite> SELECT TID, COUNT(DISTINCT Subject) num_subjects, COUNT(DISTINCT Hobby) num_hobbies, COUNT(*) total_rows FROM TEACHER GROUP BY TID;
TID | num_subjects | num_hobbies | total_rows
-----+-----
1   | 2            | 2           | 4
```

Step 2: Decompose to 4NF

- TEACHER_SUBJECT(TID, Subject)
- TEACHER_HOBBY(TID, Hobby)

Each MVD is now isolated in its own relation, eliminating the redundant cross-product of subjects and hobbies. Adding a new hobby for a teacher now means one row in TEACHER_HOBBY, not one new row per existing subject.

```
sqlite> -- Q7 (b) - decomposed to 4NF: TEACHER_SUBJECT, TEACHER_HOBBY
sqlite> CREATE TABLE TEACHER_SUBJECT (TID INTEGER, Subject TEXT, PRIMARY KEY(TID, Subject));
sqlite> INSERT INTO TEACHER_SUBJECT VALUES (1,'Maths'),(1,'Physics');

sqlite> CREATE TABLE TEACHER_HOBBY (TID INTEGER, Hobby TEXT, PRIMARY KEY(TID, Hobby));
sqlite> INSERT INTO TEACHER_HOBBY VALUES (1,'Chess'),(1,'Music');

sqlite> SELECT * FROM TEACHER_SUBJECT;
TID | Subject
-----+-----
1   | Maths
1   | Physics

sqlite> SELECT * FROM TEACHER_HOBBY;
TID | Hobby
-----+-----
1   | Chess
1   | Music
```

Q8. Join Dependencies and 5NF

Concept

A join dependency $JD(R_1, R_2, \dots, R_n)$ holds on R if R is always equal to the natural join of $R_1..R_n$. A relation can be in 4NF yet still hold redundancy caused by a join dependency that isn't implied by any candidate key — this is exactly what 5NF (Project-Join Normal Form) eliminates.

Example: SUPPLY(Agent, Company, Product)

Business rule: "if agent A represents company C, company C makes product P, and agent A sells product P somewhere, then A supplies P for C." This is a genuine 3-way (ternary) relationship that cannot be losslessly captured by any 2-way decomposition — it requires all three pairwise projections together.

- R1(Agent, Company)
- R2(Company, Product)
- R3(Agent, Product)

The demonstration below rejoins all three projections and reproduces the original SUPPLY table exactly (no spurious tuples), confirming the ternary join dependency and validating the 5NF decomposition. A naive two-table split (say, just R1 and R2) would not carry this constraint and could reintroduce a combination that never actually happened.

```
sqlite> -- Q8 - SUPPLY(Agent,Company,Product): ternary join dependency, needs 5NF
sqlite> CREATE TABLE SUPPLY (Agent TEXT, Company TEXT, Product TEXT);

sqlite> INSERT INTO SUPPLY VALUES ('A1','C1','P1'),('A1','C2','P1'),
...> ('A2','C1','P1'),('A1','C1','P2');

sqlite> SELECT * FROM SUPPLY ORDER BY Agent, Company, Product;
Agent | Company | Product
-----|-----|-----
A1    | C1      | P1
A1    | C1      | P2
A1    | C2      | P1
A2    | C1      | P1

sqlite> CREATE TABLE R_AC (Agent TEXT, Company TEXT);
sqlite> INSERT INTO R_AC SELECT DISTINCT Agent, Company FROM SUPPLY;
sqlite> CREATE TABLE R_CP (Company TEXT, Product TEXT);
sqlite> INSERT INTO R_CP SELECT DISTINCT Company, Product FROM SUPPLY;
sqlite> CREATE TABLE R_AP (Agent TEXT, Product TEXT);
sqlite> INSERT INTO R_AP SELECT DISTINCT Agent, Product FROM SUPPLY;

sqlite> -- rejoin all three pairwise projections (a 2-way split would lose or add rows)

sqlite> SELECT DISTINCT ac.Agent, ac.Company, ap.Product FROM R_AC ac JOIN R_AP ap ON ac.Agent=ap.Agent JOIN R_CP cp ON ac.Company=cp.Company AND ap.Product=cp.Product;
Agent | Company | Product
-----|-----|-----
A1    | C1      | P1
A1    | C1      | P2
A1    | C2      | P1
A2    | C1      | P1
```

Q9. Anomalies in a Poorly Designed College Schema — Normalize to 3NF

Step 1: Identify anomalies in the unnormalized design

COLLEGE(StudID, StudName, DeptID, DeptName, HOD, CourseID, CourseName) mixes student, department, and course facts into one relation.

- Insertion anomaly: a new department cannot be recorded until at least one student enrolls in one of its courses.
- Deletion anomaly: deleting the only student record for a department also erases that department's DeptName and HOD.
- Update anomaly: since DeptName/HOD are repeated on every course row for a department, updating the HOD requires touching every matching row — miss one, and the data becomes inconsistent.

The demonstration below updates HOD on only one row and shows the resulting inconsistency.

```

sqlite> -- Q9 (a) - unnormalized COLLEGE table: update anomaly demonstration
sqlite> CREATE TABLE COLLEGE_UNNORM (StudID INTEGER, StudName TEXT, DeptID TEXT, DeptName TEXT, HOD TEXT, CourseID TEXT, CourseName TEXT);

sqlite> INSERT INTO COLLEGE_UNNORM VALUES (1,'Arun','D01','CSE','Dr. Menon','CS101','DBMS'),
...> (1,'Arun','D01','CSE','Dr. Menon','CS102','OS'),
...> (2,'Bala','D02','ECE','Dr. Iyer','EC101','Signals');

sqlite> SELECT * FROM COLLEGE_UNNORM;
StudID | StudName | DeptID | DeptName | HOD | CourseID | CourseName
-----+-----+-----+-----+-----+-----+-----
1      | Arun    | D01    | CSE      | Dr. Menon | CS101    | DBMS
1      | Arun    | D01    | CSE      | Dr. Menon | CS102    | OS
2      | Bala    | D02    | ECE      | Dr. Iyer  | EC101    | Signals

sqlite> -- update anomaly: changing HOD for D01 requires updating multiple rows

sqlite> UPDATE COLLEGE_UNNORM SET HOD='Dr. Nair' WHERE DeptID='D01' AND CourseID='CS101';

sqlite> SELECT StudID, DeptID, HOD FROM COLLEGE_UNNORM WHERE DeptID='D01';
StudID | DeptID | HOD
-----+-----+-----
1      | D01    | Dr. Nair
1      | D01    | Dr. Menon

sqlite> -- HOD is now inconsistent between the two CSE rows -> update anomaly

```

Step 2: Normalize to 3NF

- STUDENT(StudID, StudName, DeptID)
- DEPARTMENT(DeptID, DeptName, HOD)
- COURSE(CourseID, CourseName, DeptID)
- ENROLL(StudID, CourseID) — resolves the many-to-many student↔course relationship

HOD is now stored exactly once per department, so a single UPDATE on DEPARTMENT fixes it everywhere with no inconsistency, as shown below. The ENROLL table also fixes the insertion anomaly, since a department and its c

```

sqlite> -- Q9 (b) - normalized to 3NF: STUDENT, DEPARTMENT, COURSE, ENROLL
sqlite> CREATE TABLE STUDENT_9 (StudID INTEGER PRIMARY KEY, StudName TEXT, DeptID TEXT);
sqlite> INSERT INTO STUDENT_9 VALUES (1,'Arun','D01'),(2,'Bala','D02');

sqlite> CREATE TABLE DEPT_9 (DeptID TEXT PRIMARY KEY, DeptName TEXT, HOD TEXT);
sqlite> INSERT INTO DEPT_9 VALUES ('D01','CSE','Dr. Menon'),('D02','ECE','Dr. Iyer');

sqlite> CREATE TABLE COURSE_9 (CourseID TEXT PRIMARY KEY, CourseName TEXT, DeptID TEXT);
sqlite> INSERT INTO COURSE_9 VALUES ('CS101','DBMS','D01'),('CS102','OS','D01'),('EC101','Signals','D02');

sqlite> CREATE TABLE ENROLL_9 (StudID INTEGER, CourseID TEXT, PRIMARY KEY(StudID, CourseID));
sqlite> INSERT INTO ENROLL_9 VALUES (1,'CS101'),(1,'CS102'),(2,'EC101');

sqlite> SELECT * FROM STUDENT_9;
StudID | StudName | DeptID
-----+-----+-----
1      | Arun    | D01
2      | Bala    | D02

sqlite> SELECT * FROM DEPT_9;
DeptID | DeptName | HOD
-----+-----+-----
D01    | CSE      | Dr. Menon
D02    | ECE      | Dr. Iyer

sqlite> SELECT * FROM COURSE_9;
CourseID | CourseName | DeptID
-----+-----+-----
CS101    | DBMS       | D01
CS102    | OS         | D01
EC101    | Signals    | D02

sqlite> SELECT * FROM ENROLL_9;
StudID | CourseID
-----+-----
1      | CS101
1      | CS102
2      | EC101

sqlite> -- HOD is stored once per department now: one UPDATE fixes it everywhere

sqlite> UPDATE DEPT_9 SET HOD='Dr. Nair' WHERE DeptID='D01';

sqlite> SELECT * FROM DEPT_9;
DeptID | DeptName | HOD
-----+-----+-----
D01    | CSE      | Dr. Nair
D02    | ECE      | Dr. Iyer

```

Q10. Attribute Closure and Candidate Keys

Relation and FDs

$R(A,B,C,D,E)$ with FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$

Closure algorithm:

For each attribute set X , start $\text{result} = X$, then repeatedly apply any FD whose left side is already inside result , adding its right side, until no more attributes can be added.

Computing the closure of every single attribute shows that only A^+ expands (via the chain $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$) to cover all five attributes, so only A is a superkey among the singletons; the others' closures fall short.

```
>>> python3 closure_algorithm.py

Relation R(A,B,C,D,E)
FDs given: A->B, B->C, C->D, D->E

Computing A+ :
  Apply A -> B : A => AB
  Apply B -> C : AB => ABC
  Apply C -> D : ABC => ABCD
  Apply D -> E : ABCD => ABCDE
  Result: A+ = {A,B,C,D,E}
  Superkey? YES

Computing B+ :
  Apply B -> C : B => BC
  Apply C -> D : BC => BCD
  Apply D -> E : BCD => BCDE
  Result: B+ = {B,C,D,E}
  Superkey? no

Computing C+ :
  Apply C -> D : C => CD
  Apply D -> E : CD => CDE
  Result: C+ = {C,D,E}
  Superkey? no

Computing D+ :
  Apply D -> E : D => DE
  Result: D+ = {D,E}
  Superkey? no

Computing E+ :
  No FD applicable; E+ = {E}
  Result: E+ = {E}
  Superkey? no

Candidate key check: A+ = {A,B,C,D,E} = all attributes => A is a candidate key
No proper subset of a single attribute exists below A, so A is minimal.
CANDIDATE KEY(S) = { A }
```

Conclusion:

Candidate Key = $\{ A \}$. Since $A^+ = \{A,B,C,D,E\}$ (all attributes) and no proper subset of $\{A\}$ (i.e. the empty set) achieves this, A is both a superkey and minimal — hence a candidate key. B, C, D, E are non-prime attributes. Because the FDs form a single chain ($A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$), this relation also happens to already satisfy 3NF and BCNF, with no further decomposition needed.